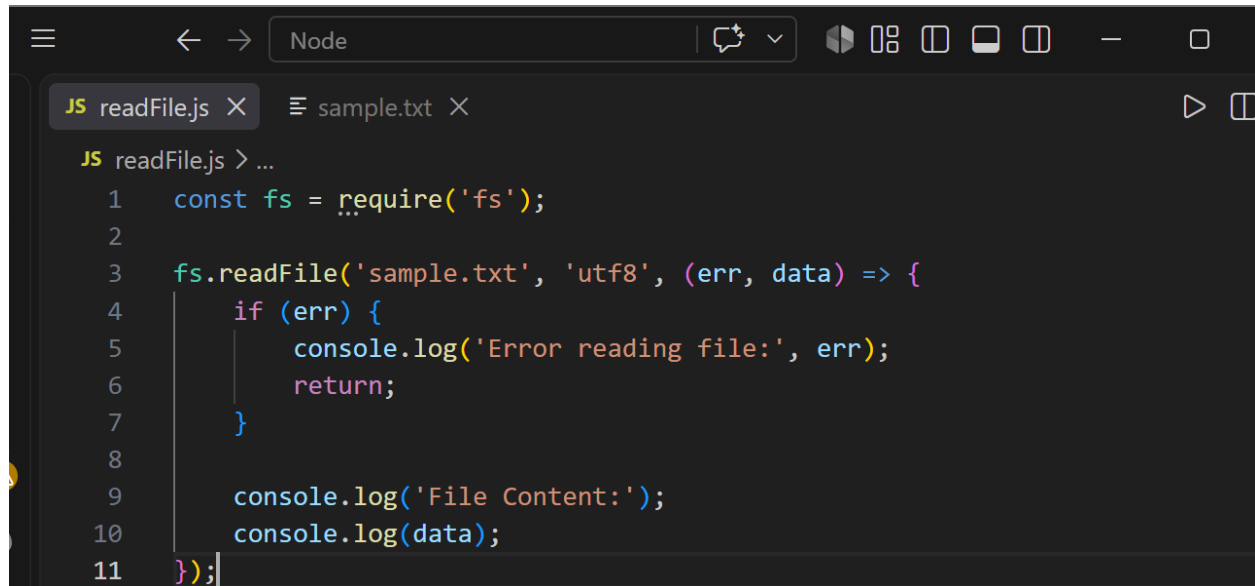


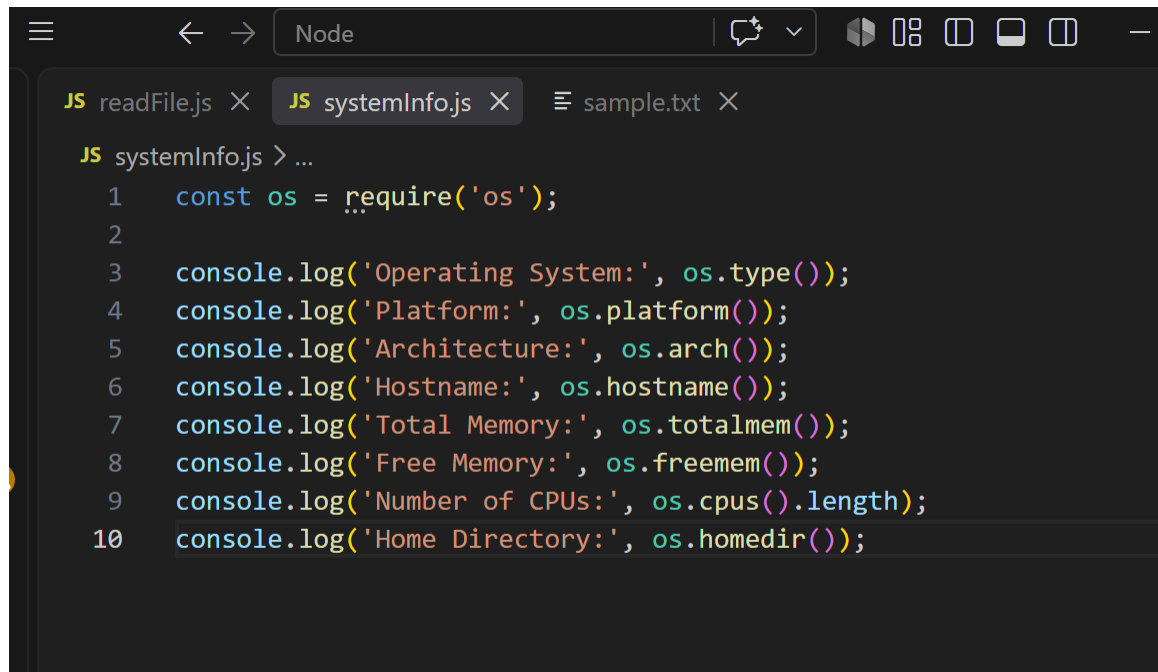
Ques 1: Construct a program to demonstrate usage of Node.js fs module to read a file.



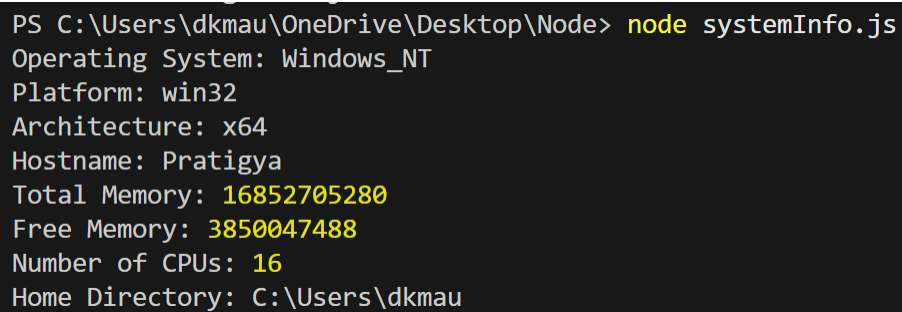
```
JS readFile.js X sample.txt X
JS readFile.js > ...
1  const fs = require('fs');
2
3  fs.readFile('sample.txt', 'utf8', (err, data) => {
4      if (err) {
5          console.log('Error reading file:', err);
6          return;
7      }
8
9      console.log('File Content:');
10     console.log(data);
11 });
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node readFile.js
File Content:
Hello, this is a sample file.
We are learning Node.js fs module.
PS C:\Users\dkmau\OneDrive\Desktop\Node> 
```

Ques 2:Construct a program to display system information using the os module.

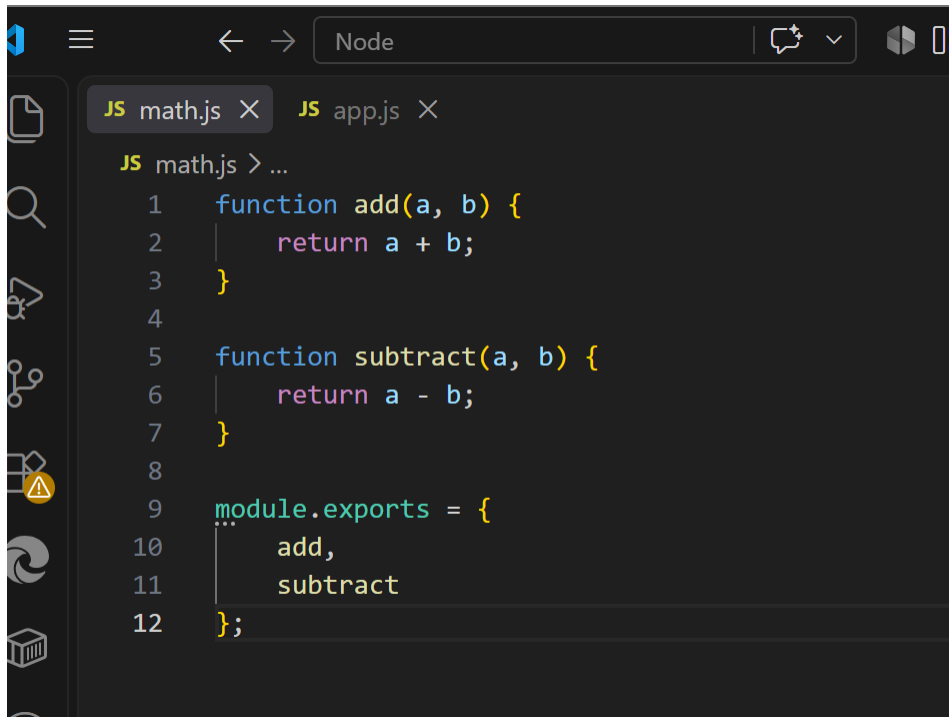
A screenshot of a code editor window with a dark theme. The window has a title bar with a hamburger menu, navigation arrows, and the text 'Node'. Below the title bar, there are three tabs: 'JS readFile.js', 'JS systemInfo.js' (which is active), and 'sample.txt'. The code in the active tab is as follows:

```
JS systemInfo.js > ...
1  const os = require('os');
2
3  console.log('Operating System:', os.type());
4  console.log('Platform:', os.platform());
5  console.log('Architecture:', os.arch());
6  console.log('Hostname:', os.hostname());
7  console.log('Total Memory:', os.totalmem());
8  console.log('Free Memory:', os.freemem());
9  console.log('Number of CPUs:', os.cpus().length);
10 console.log('Home Directory:', os.homedir());
```

A screenshot of a terminal window with a dark background. The prompt is 'PS C:\Users\dkmau\OneDrive\Desktop\Node>'. The command 'node systemInfo.js' has been executed, and the output is displayed as follows:

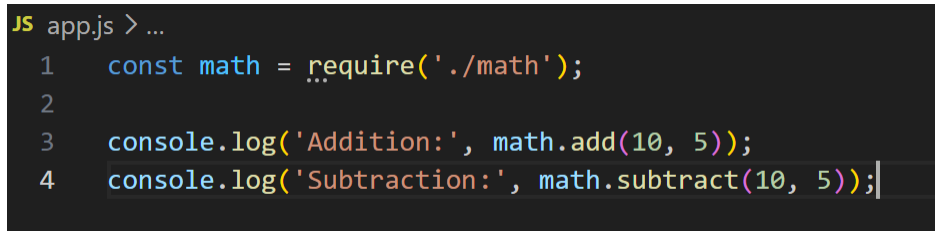
```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node systemInfo.js
Operating System: Windows_NT
Platform: win32
Architecture: x64
Hostname: Pratigya
Total Memory: 16852705280
Free Memory: 3850047488
Number of CPUs: 16
Home Directory: C:\Users\dkmau
```

Ques 3:Construct a program to create and export a simple Node.js module.



The screenshot shows the Visual Studio Code editor with two tabs: 'math.js' and 'app.js'. The 'math.js' tab is active, displaying the following JavaScript code:

```
JS math.js > ...
1  function add(a, b) {
2      return a + b;
3  }
4
5  function subtract(a, b) {
6      return a - b;
7  }
8
9  module.exports = {
10     add,
11     subtract
12 };
```



The screenshot shows the Visual Studio Code editor with the 'app.js' tab active, displaying the following JavaScript code:

```
JS app.js > ...
1  const math = require('./math');
2
3  console.log('Addition:', math.add(10, 5));
4  console.log('Subtraction:', math.subtract(10, 5));
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node app.js
Addition: 15
Subtraction: 5
```

Ques 4: Construct a program to parse a URL using the Node.js url module.

```
JS parseUrl.js X
JS parseUrl.js > ...
1  const url = require('url');
2
3  const myUrl = 'https://www.example.com:8080/products?category=mobile&id=101';
4
5  const parsedUrl = url.parse(myUrl, true);
6
7  console.log('Protocol:', parsedUrl.protocol);
8  console.log('Host:', parsedUrl.host);
9  console.log('Hostname:', parsedUrl.hostname);
10 console.log('Port:', parsedUrl.port);
11 console.log('Path:', parsedUrl.pathname);
12 console.log('Query:', parsedUrl.query);

PS C:\Users\dkmau\OneDrive\Desktop\Node> node parseUrl.js
Protocol: https:
Host: www.example.com:8080
Hostname: www.example.com
Port: 8080
Path: /products
Query: [Object: null prototype] { category: 'mobile', id: '101' }
(node:19264) [DEP0169] DeprecationWarning: 'url.parse()' behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for 'url.parse()' vulnerabilities.
(Use 'node --trace-deprecation ...' to show where the warning was created)
```

Ques 5:Construct a program to create a basic HTTP server using Node.js.

```
JS server.js X
C:\Users\dkmau\OneDrive\Desktop\Node\server.js
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4    res.writeHead(200, {
5      'Content-Type': 'text/plain'
6    });
7
8    res.end('Hello! Welcome to my Node.js HTTP Server.');
```

```
9  });
10
11  server.listen(3000, () => {
12    console.log('Server running at http://localhost:3000/');
13  });
```

← → ↻ http://localhost:3000/

Hello! Welcome to my Node.js HTTP Server.

Ques 6:Construct a program to demonstrate callback-based asynchronous execution.

```
JS server.js X JS callback.js X
JS callback.js > ...
1  function performTask(callback) {
2      setTimeout(() => {
3          console.log('Task completed');
4          callback();
5      }, 2000);
6  }
7
8  console.log('Task started');
9
10 performTask(() => {
11     console.log('Callback executed');
12 });
13
14 console.log('Task is running asynchronously');
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node callback.js
Task started
Task is running asynchronously
Task completed
Callback executed
```

Ques 7:Construct a program to illustrate promise creation and resolution.

```
JS promise.js X
JS promise.js > ...
1  const myPromise = new Promise((resolve, reject) => {
2      const success = true;
3
4      if (success) {
5          resolve('Promise resolved successfully!');
6      } else {
7          reject('Promise rejected!');
8      }
9  });
10
11  myPromise
12      .then((result) => {
13          console.log(result);
14      })
15      .catch((error) => {
16          console.log(error);
17      });
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node promise.js
Promise resolved successfully!
```

Ques 8:Construct a program to demonstrate promise chaining with multiple operations.

```
JS promise.js X JS promiseChain.js X
JS promiseChain.js > ...
1  function step1() {
2    |   return Promise.resolve(10);
3    | }
4
5  function step2(value) {
6    |   return Promise.resolve(value + 20);
7    | }
8
9  function step3(value) {
10   |   return Promise.resolve(value * 2);
11   | }
12
13  step1()
14    .then((result) => {
15      |   console.log('Step 1:', result);
16      |   return step2(result);
17      | })
18    .then((result) => {
19      |   console.log('Step 2:', result);
20      |   return step3(result);
21      | })
22    .then((result) => {
23      |   console.log('Step 3:', result);
24      | })
25    .catch((error) => {
26      |   console.log('Error:', error);
27      | });
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node promiseChain.js
Step 1: 10
Step 2: 30
Step 3: 60
```

Ques 9:Construct a program to show JSON data parsing and stringifying.

```
JS jason.js > ...
1  const student = {
2      name: 'Pratigya',
3      age: 20,
4      course: 'B.Tech CSE',
5      skills: ['Java', 'JavaScript', 'Node.js']
6  };
7
8  // Convert JavaScript object into JSON string
9  const jsonString = JSON.stringify(student);
10
11 console.log('JSON String:');
12 console.log(jsonString);
13
14 // Convert JSON string back into JavaScript object
15 const jsonObject = JSON.parse(jsonString);
16
17 console.log('\nJavaScript Object:');
18 console.log(jsonObject);
19
20 console.log('\nStudent Name:', jsonObject.name);
21 console.log('Course:', jsonObject.course);
```

```
PS C:\Users\dkmau\OneDrive\Desktop\Node> node jason.js
JSON String:
{"name":"Pratigya","age":20,"course":"B.Tech CSE","skills":["Java","JavaScript","Node.js"]}

JavaScript Object:
{
  name: 'Pratigya',
  age: 20,
  course: 'B.Tech CSE',
  skills: [ 'Java', 'JavaScript', 'Node.js' ]
}

Student Name: Pratigya
Course: B.Tech CSE
```

Ques 10: Construct a program to demonstrate routing in an Express application.

```
Node
JS promise.js X JS promiseChain.js X JS json.js X JS app1.js X
express-routing > JS app1.js > ...
1  const express = require('express');
2
3  const app = express();
4
5  const PORT = 3000;
6
7  // Home route
8  app.get('/', (req, res) => {
9    res.send('Welcome to the Home Page');
10 });
11
12 // About route
13 app.get('/about', (req, res) => {
14   res.send('This is the About Page');
15 });
16
17 // Contact route
18 app.get('/contact', (req, res) => {
19   res.send('This is the Contact Page');
20 });
21
22 // Student route
23 app.get('/student', (req, res) => {
24   res.send('Welcome to the Student Page');
25 });
26
27 // Start server
28 app.listen(PORT, () => {
29   console.log(`Server running at http://localhost:${PORT}`);
30 });
```



Welcome to the Home Page